DESIGN AND ANALYSIS OF ALGORITHMS

Project Report

Lightpath: Latency-Aware Cloud Task Placement on Network Graphs

Graph Algorithms, Dynamic Programming and Greedy Scheduling for Replica and VM Placement



Team Members

Naman Singla – 25WU0101084

Mohak Gupta – 25WU0101074

Faculty

Dr. Monday Jubrin Abdullahi

Section : CSE RHINOS

Academic Year : 2026–2027

Table of Contents

1 Introduction and Problem Statement **1**

2 Problem Formulation **2**

3 Technology Stack and Justification **3**

4 Algorithms Used, and Why Each One Was Chosen **4**

5 System Design (UML) **8**

6 Implementation and Working Screenshots **11**

7 Complexity Analysis **13**

8 Testing and Demonstrated Scenarios **13**

9 Experimental Evaluation **14**

10 Case Studies and Policy Outputs **18**

11 Correctness Reasoning (Proof Sketches) **19**

12 Limitations and Future Work **19**

13 Conclusion **19**

References **20**

1 Introduction and Problem Statement

1.1 Course context

This report documents *Lightpath*, our project for the Design and Analysis of Algorithms (DAA) course at Woxsen University. Of the problem statements offered in the brief, we chose **Cloud Task Placement**: servers are nodes, link latencies are edge weights, and replicas or virtual machines (VMs) must be placed to minimise average latency while respecting server capacity. The brief asks for minimum spanning trees for the backbone, shortest paths for routing and dynamic programming for placement selection, together with greedy schedulers, experiments that vary size, density and capacity, and policy outputs. Lightpath implements every item on that list and adds an exact optimiser so that the quality of each heuristic can be measured rather than assumed.

1.2 Motivation: why placement is a real algorithmic problem

A request to an online service, whether a video stream, a payment or a search, travels from the user's nearest network site to a server that holds a copy of the service. The delay the user feels is dominated by the distance the signal travels through fibre and the number of switches it crosses. Putting copies close to users makes them fast, but three concerns pull against each other:

- **Latency.** Every user should reach a nearby copy. This pushes copies towards the cities with the most demand.
- **Capacity.** Each server has limited CPU and memory. A popular city cannot host every service, and a server used by one service may be needed by another.
- **Cost.** Every copy costs money per hour, and every server that is switched on has a running cost. The budget limits how many copies we can buy.

Deciding *where to run what* under these pressures is a combinatorial optimisation problem on a graph. It touches almost every family of algorithms in the course: shortest paths to measure distance, spanning trees to design the network, dynamic programming to spend a budget, and greedy methods to make fast decisions when exact search is too slow.

1.3 What makes it hard

If there were only one service, no capacity limits and a fixed number of copies, the problem would be exactly the classical *p-median* problem: choose p sites so that the total distance from every client to its nearest chosen site is as small as possible. The p-median problem is NP-hard on general graphs [1], and adding capacities adds bin-packing structure, which is NP-hard by itself. No known algorithm solves every instance exactly in polynomial time. Our approach therefore uses exact algorithms wherever a sub-problem is tractable, fast greedy heuristics for the hard placement step, and an exact solver on smaller networks to measure how far the heuristics are from the true optimum.

1.4 Objectives and contributions

The project set out to answer three questions:

Q1 Which classical algorithm is the right one for each stage, and do their theoretical bounds match what we measure?

Q2 How close do fast greedy schedulers get to the true optimum, and what does closing the gap cost?

Q3 How do network size, density, server capacity and budget change the answer?

Our contributions are: a seven-stage pipeline that solves capacitated, budgeted, multi-service placement; implementations of every algorithm from first principles (including four priority queues and union-

find); a transfer-cost model with negative edges that motivates Bellman-Ford and Johnson; ten test networks including India, Nigeria and the Woxsen campus; eight experiments with an exact reference; 191 automated tests; and an interactive website that explains and demonstrates all of it.

2 Problem Formulation

2.1 Inputs

The cloud is modelled as a weighted, undirected graph $G = (V, E)$:

- **Nodes** V are network sites. Most are servers with a CPU capacity (vCPU), a memory capacity (GB) and an hourly running cost. Some are routers that forward traffic but cannot host anything.
- **Edges** E are fibre links. Each link carries a **latency** in milliseconds and a bandwidth. For networks with real coordinates, latency is computed from distance: light in fibre covers about 200 km per millisecond, cables are about 30 % longer than the straight line, and each hop adds about 0.3 ms of switching.
- **Services** are the workloads, such as web front end, video transcoding or payments. Each needs a fixed amount of CPU and memory per copy, costs a fixed amount per copy-hour, has a priority, a minimum and maximum number of copies, and a **demand**: how many requests per second each city sends to it.
- **Budgets** limit the total hourly spend on copies and, optionally, on switching servers on.

Storage is an adjacency list, so memory grows with the number of nodes plus links rather than with the square of the number of nodes. Real networks are sparse, with each city linked to only a few others, so this matters.

2.2 Objective

Every user is served by the nearest copy of the service they request, measured along the shortest path in the network. The quantity we minimise is the **demand-weighted average latency**:

$$\text{average latency} = \frac{\sum (\text{requests from city } c \text{ for service } j) \times (\text{distance from } c \text{ to the nearest copy of } j)}{\text{total requests}}.$$

Requests from higher-priority services count more. We also report the 95th-percentile and worst-case latency, because an average can hide a few badly served cities.

2.3 Constraints

A plan is valid only if:

- every service has between its minimum and maximum number of copies, and at most one copy per server (so one server failure cannot take out two copies);
- on every server, the CPU and memory used by the copies it hosts do not exceed its capacity;
- the total hourly cost of all copies stays within the VM budget, and if a server budget is set, only servers bought within it are used.

2.4 Transfer costs and negative edges

Copies of the same service must synchronise data. Moving a gigabyte from site u to site v costs a small base fee plus a per-millisecond charge, adjusted by the difference in regional price levels between u and v. Core data centres are cheaper than edge sites, so sending data towards a more expensive region earns a *rebate*. Some arcs therefore have a **negative cost**: in the India network, 24 of 88 directed arcs are negative.

Negative edges break Dijkstra's algorithm, which is why the brief's Bellman-Ford and Johnson algorithms have a genuine role here. Negative edges are only dangerous if they form a *negative cycle*, because then the cost could be lowered for ever by going round the loop. Our cost model rules this out

(Lemma 1, Section 11).

2.5 Infeasibility handling

The system reports problems instead of crashing or silently returning a wrong plan:

- **Invalid network.** Before solving, the instance is validated: links must have positive latency, services must have valid replica ranges, and every demand must come from a real site. Disconnected networks are flagged, and the spanning tree becomes a spanning forest.
- **Not enough capacity.** If the minimum copies cannot all fit, the sizing stage reports it, and strategies that cannot place every copy are marked *infeasible* with the reason. They are never shown as valid plans.
- **A service no server can host.** Such a service is kept at zero copies and the same penalty is applied in both the heuristics and the exact solver, so their scores remain comparable.

3 Technology Stack and Justification

Lightpath has three layers: a Python engine that contains every algorithm, a small web API, and a browser dashboard. Table 1 gives each tool and the reason it was chosen.

Table 1: Technology stack and the rationale for each choice.

Tool	Role in the project	Why it was chosen
Python 3 (`cloudplace` package)	All algorithms, the problem model, data generators, the solver and the experiments	Readable enough that every algorithm can be written from first principles and checked line by line against the textbook. Graphs, heaps, union-find, the DPs and the greedy methods are our own code, not library calls.
NumPy	Vectorised Floyd–Warshall and the distance matrices	One vectorised version of Floyd–Warshall runs the same $\Theta(n^3)$ algorithm much faster than pure Python. That made Experiment E2 possible, comparing constant factors with asymptotics.
SciPy (HiGHS solver)	Exact MILP used only as a referee	An independent, proven-optimal answer is the only honest way to measure a heuristic's gap. It checks our algorithms and is never used to produce them.
FastAPI + Uvicorn	REST API serving datasets, solving and step-by-step traces	Typed request models catch bad input early, and one process serves both the API and the website.
React 18 + TypeScript + Vite	Interactive dashboard	State-driven rendering suits an app where one change (network, budget, start city) must update the map, tables and charts together. TypeScript catches data-shape errors between engine and UI.
Custom SVG map	Network drawing, routes, step animations	Full control over highlighting exactly the edge or node each algorithm touches at each step, without a heavy graph library.
pytest, Hypothesis, NetworkX	191 automated tests	NetworkX is an independent reference for every graph algorithm; Hypothesis generates random inputs for heap tests.

Why a working application and not only a script. The brief asks for a dashboard and policy outputs. An interactive site also makes the algorithms visible: in the Algorithm Lab a viewer can watch Dijkstra settle cities one by one, or see Bellman-Ford handle a rebate link that would fool Dijkstra. Running on saved results, the site also works offline, so it can be demonstrated without installing Python.

4 Algorithms Used, and Why Each One Was Chosen

This section is the analytical core of the report. For each stage it states what the stage must achieve, which algorithm was selected, why that algorithm fits, and which alternatives were set aside. Fig. 1 shows how the stages connect: each one answers one question, and its output feeds the next.

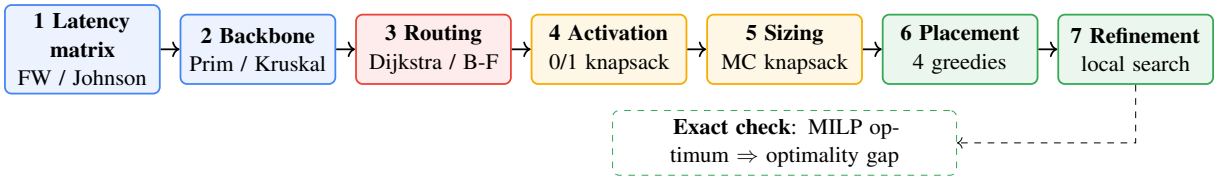


Figure 1: The Lightpath pipeline. Stages 1–3 are exact polynomial algorithms, 4–5 exact dynamic programs, and 6–7 fast heuristics whose distance from the optimum is measured by the exact MILP.

4.1 Supporting data structures

4.1.1 Adjacency list

Each site keeps a list of its neighbours with the link's latency, bandwidth and cost. Memory is proportional to sites plus links, and visiting a site's neighbours costs only its number of links. An adjacency matrix was rejected: it wastes n^2 cells on links that do not exist and makes every neighbour scan cost n.

4.1.2 Four priority queues

Dijkstra and Prim repeatedly need the closest unfinished site, which is a priority-queue job. We implemented four queues behind one interface (*push*, *pop-minimum*, *decrease-key*) so the same algorithm can run on any of them:

- **Binary heap** with a position map: every operation in $O(\log n)$.
- **Fibonacci heap** [2]: insert and decrease-key in amortised $O(1)$, extract-min in $O(\log n)$. It is the textbook way to get the best bound for Dijkstra and Prim.
- **Pairing heap**: simpler than Fibonacci with good practical speed.
- **Lazy heap** (Python's built-in `heapq`): instead of decreasing a key, it pushes a new entry and skips stale ones when they surface.

Implementing all four was a deliberate choice: Experiment E1 tests whether the heap with the best Big-O is also the fastest in practice. It is not.

4.1.3 Union-find

Kruskal must check quickly whether a link would close a loop. Union-find with *union by rank* and *path compression* does this in almost constant amortised time, $O(\alpha(n))$ per operation [3].

4.2 Minimum spanning tree: Prim's and Kruskal's algorithms

4.2.1 Why an MST at all?

Every physical link costs money to build and maintain. The cheapest way to connect every site with no redundant link is the **minimum spanning tree**: a set of $n-1$ links that connects all sites with the smallest total latency. In Lightpath the MST is the *provisioning backbone*, the first set of links an operator would build. It is also used to measure *stretch*: how much slower requests would be if they could only travel along the backbone.

4.2.2 Why implement both Prim and Kruskal

Both find the same cheapest tree but in different ways, and each wins in a different situation.

- **Prim** [4] grows a single tree from a start city, always adding the cheapest link that reaches a new city. Using our heaps it runs in $O(E \log V)$, or $O(E + V \log V)$ with a Fibonacci heap. It never sorts the whole edge list, which makes it better on *dense* graphs.
- **Kruskal** [5] sorts all links from cheapest to most expensive and keeps each one unless union-find reports that it would close a loop. Its cost is dominated by the sort, $O(E \log E)$, which is small on *sparse* graphs.

The pipeline chooses automatically: **Prim if $E > V \log_2 V$, otherwise Kruskal**. Experiment E3 confirms the rule. On sparse graphs Kruskal took 1.5 ms against Prim's 3.7 ms; on dense graphs Prim took 42 ms against Kruskal's 126 ms. Both always returned the same tree weight.

4.2.3 Alternatives considered

Borůvka's algorithm grows many fragments in parallel. Its advantage is parallelism, which a single-threaded Python program does not use, so we did not implement it. **Steiner trees** connect only a chosen subset of sites. They do not match our assumption that every site belongs to the backbone.

4.3 Single-source shortest paths: Dijkstra and Bellman-Ford

4.3.1 Role in the pipeline

Once copies are placed, every request must be *routed*: we need the actual path from each city to its nearest copy, hop by hop, for the route table that operators use. Copies of the same service must also synchronise, and that traffic should take the *cheapest* route in dollars per gigabyte. These are two single-source shortest-path problems with different weights.

4.3.2 Dijkstra for latency

Dijkstra's algorithm [6] repeatedly takes the closest unsettled city from a priority queue and declares its distance final, then updates its neighbours. Latency can never be negative, so Dijkstra is correct and fast: $O((V + E) \log V)$ with a binary heap. In the Algorithm Lab it also demonstrates an important property: if only one destination matters, Dijkstra can *stop* as soon as that destination is settled, because its distance can no longer change.

4.3.3 Bellman-Ford for transfer cost

For dollars per gigabyte, some links are rebates with negative cost. Dijkstra can then give wrong answers, because a city it has already settled might be reached more cheaply later through a negative link. **Bellman-Ford** [7] relaxes every link in rounds. After round i, every route that uses at most i links is correct, so $V - 1$ rounds are enough. Any improvement in round V would reveal a negative cycle, which our implementation detects and reports. It is slower, $O(VE)$, but it is the right tool when weights can be negative. In Lightpath it computes the cheapest replica-synchronisation routes.

4.3.4 Why both, and not one

Using Bellman-Ford everywhere would be correct but needlessly slow for latency, at 490 ms against 144 ms at 4000 sites in E1. Using Dijkstra everywhere would be fast but wrong for costs with rebates. Each is used exactly where its guarantee holds.

4.4 All-pairs shortest paths: Floyd–Warshall and Johnson

4.4.1 Role in the pipeline

Placement must try many candidate servers for every service, and each trial needs the distance from every city to every server. Computing those distances once, as a full **latency matrix** with an entry for every pair of sites, makes each trial a cheap table lookup. This matrix is the input to every later stage.

4.4.2 Floyd–Warshall

Floyd–Warshall [8] is dynamic programming over intermediate cities. The best route from i to j that may pass through cities $1..k$ either avoids city k or goes through it once, so the table is updated with $d[i][j] = \min(d[i][j],\ d[i][k] + d[k][j])$ for every k. It runs in $\Theta(V^3)$ time and handles negative edges. Its triple loop is also easy to vectorise: our NumPy version runs the same algorithm many times faster than pure Python.

4.4.3 Johnson

Johnson's algorithm [9] targets sparse graphs with negative edges. It runs Bellman-Ford once from an added helper vertex to get a *potential* h for every city, then reweights every link to $w + h(u) - h(v)$, which is never negative. Dijkstra can then run safely from every city. Every route between the same two cities changes by the same amount, so shortest routes stay shortest. The total is $O(VE + V(V+E)\log V)$, better than V^3 when the graph is sparse.

4.4.4 The choice rule, and what the experiment taught us

The pipeline treats a graph as sparse when $E \log V < V^2$ and then uses repeated Dijkstra (or Johnson if any weight is negative); dense graphs use Floyd–Warshall. Experiment E2 tested this rule. On dense graphs it was clearly right: at 400 sites, NumPy Floyd–Warshall took 277 ms against Johnson's 5015 ms. On sparse graphs, Big-O favours Johnson, yet NumPy Floyd–Warshall was still fastest at this size (285 ms against 985 ms). The lesson is that *constant factors and implementation matter as much as asymptotics*, so a choice rule should be calibrated by measurement.

4.5 Dynamic programming: two knapsack problems

4.5.1 Why dynamic programming here

Two budget decisions have exactly the structure of knapsack problems: a set of items with costs and values, a fixed budget, and the goal of the most value within budget. A greedy rule such as "best value per dollar first" can be fooled: a cheap item may block a better combination. Dynamic programming considers every budget level systematically and is guaranteed optimal. Budgets are in steps of \$0.01, so the tables stay small enough to be fast.

4.5.2 0/1 knapsack: which servers to switch on

When a server budget is set, each server is either on or off. Its *value* is the latency saving it offers and its *weight* is its hourly cost. The DP table $\mathrm{OPT}[i][w]$ holds the best value using the first i servers with budget w: server i is either skipped, keeping $\mathrm{OPT}[i-1][w]$, or taken, giving its value plus $\mathrm{OPT}[i-1][w-\mathrm{cost}_i]$. Time and space are $\Theta(nW)$, where W is the budget in cents. Experiment E8 measured about 97 ns per table cell, growing exactly linearly as predicted.

4.5.3 Multiple-choice knapsack: how many copies of each service

Each service must take *exactly one* option from its group: 1 copy, 2 copies, 3 copies and so on, each with a cost and an estimated latency saving. This is the multiple-choice knapsack problem. The DP processes one service at a time and, for every budget level, picks the best option for that service plus the best result for the earlier services with the remaining budget. Because the savings are estimated without capacity limits, a *capacity repair* step then removes the largest extra copies until first-fit-decreasing packing confirms that everything fits. This guarantees that a feasible placement exists before the placement stage starts. Experiment E7 shows the payoff: raising the India budget from \$2.41 to \$7.51 per hour cut average latency from 4.34 ms to 1.28 ms.

4.5.4 Alternatives considered

Solving the whole placement as one giant DP over every server's remaining capacity would be exact, but its table grows with the *product* of all server capacities. That is impossible for networks of 25 to 150 sites. We therefore use DP where its tables are small (budgets) and greedy methods where they would explode (placement).

4.6 Greedy schedulers for placement

4.6.1 Why greedy

Choosing the exact server for every copy is the NP-hard heart of the problem. A greedy method makes one good local choice at a time, never undoes it, and finishes in polynomial time. We built four greedy schedulers because each optimises a different concern, and the experiments compare them.

- **Demand-first.** Services with the most traffic choose first, and each copy goes to the free server nearest the next demand hotspot. Simple and intuitive.
- **Marginal gain (CELF).** At every step, add the single copy that reduces total latency the most. Benefits shrink as more copies open (the function is *submodular*), so old estimates can be re-checked lazily instead of recomputed (CELF). Without capacity limits this greedy achieves at least $1 - 1/e \approx 63\,\%$ of the best possible saving [10].
- **Regret.** Place first the service that would lose the most if it missed its best server, which is the gap between its best and second-best options. This protects services with few good choices from being crowded out.
- **FFD packing.** First-fit decreasing: place the largest copies first, each on a well-located server that fits. It focuses on *fitting* capacity rather than latency; plain FFD uses at most $\frac{11}{9}\mathrm{OPT} + \frac{6}{9}$ bins.

4.7 Local search refinement

Greedy methods never revisit a decision, so they can lock in early mistakes. Local search repeatedly tries two kinds of move: **relocate** (move one copy to a free server) and **swap** (exchange copies of two services

between two servers when both fit). Any move that lowers average latency is kept. Every accepted move strictly improves the plan and there are finitely many plans, so the search always stops. For the uncapacitated version, single-swap local optima are within a factor of 5 of optimal; in practice we observe about 0.5 %. Local search is what turns 9–38 % greedy gaps into near-optimal plans.

4.8 The exact referee: a mixed-integer program

To know whether a heuristic is good, we need the true optimum. On networks small enough, Lightpath formulates placement as a **mixed-integer linear program** (MILP): binary variables say which server hosts which copy, and the constraints are exactly those of Section 2. It is solved with the open-source HiGHS solver [11] through SciPy. When the program is too large, we report its linear relaxation as a lower bound instead. The MILP is a *checking tool*: every stage of our own pipeline is implemented by hand, and the MILP only tells us how far each heuristic is from the best possible answer.

4.9 Batch scheduling

Background batch jobs, such as nightly analytics, are scheduled with the classic greedy **LPT** rule: longest job first, each assigned to the server lane where it will finish earliest once its input data arrives over the network. Without transfer delays, LPT is within $\frac{4}{3}$ of the optimal finishing time.

5 System Design (UML)

This section describes Lightpath in standard UML views: who uses the system (use cases), how it is split into deployable parts (components), the main classes and their relationships (class diagram), what happens during one solve request (sequence diagram), and the decision flow of the solver (activity diagram).

5.1 Use-case diagram

Fig. 2 shows the three actors. A *network planner* uses the studio to load or generate networks and obtain placement policies. A *student or instructor* uses the algorithm lab and experiment pages to study the algorithms. A *researcher* runs the experiment suite and exports data. All actions go through the dashboard or the Python API.

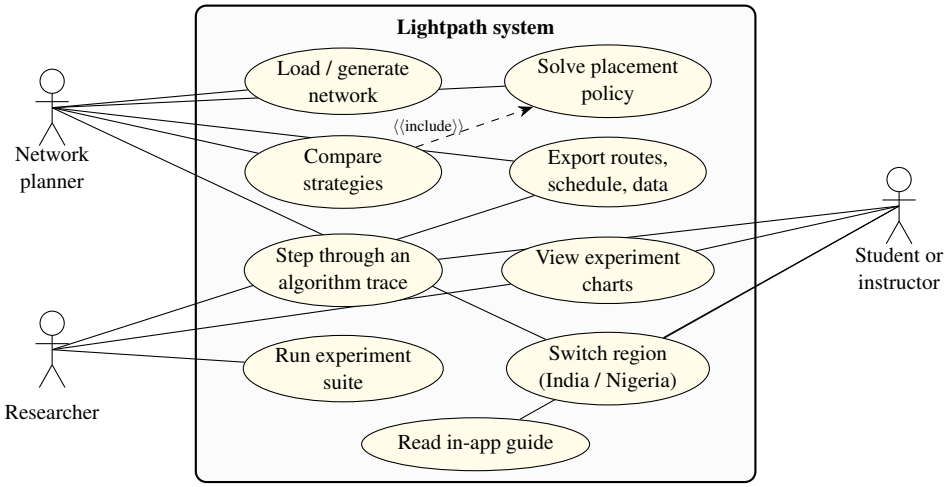


Figure 2: Use-case diagram: three actors and the nine main use cases.

5.2 Component and deployment view

Lightpath has four layers (Fig. 3). The React dashboard runs in the browser. It talks over HTTP/JSON to a FastAPI server, which also serves the built web files. The server calls the `cloudplace` solver, which

uses the algorithm and core packages. When no server is running, the dashboard reads pre-computed snapshot files instead, so the same build also works offline.

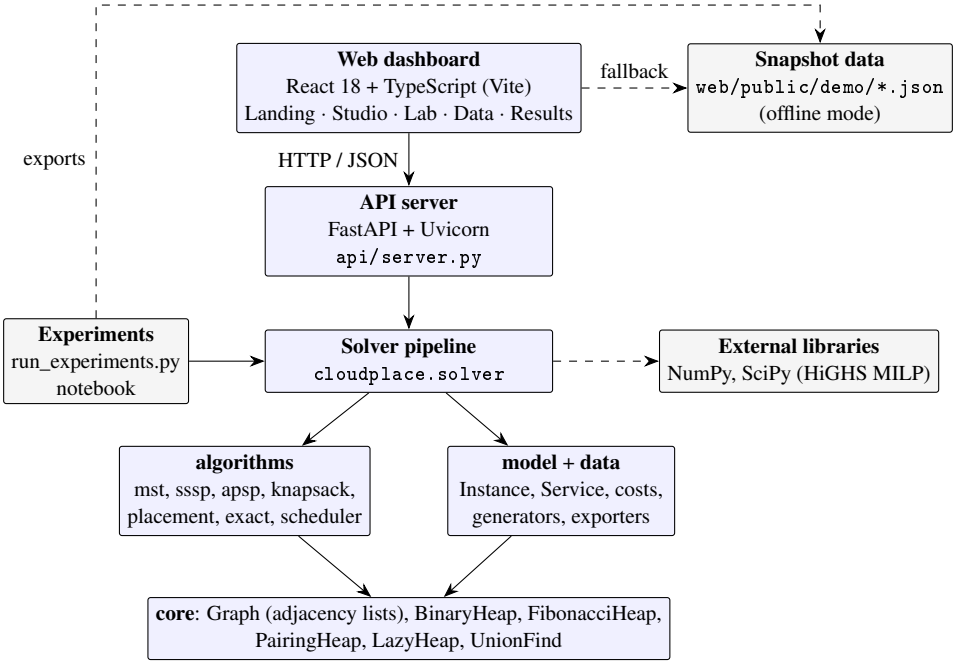


Figure 3: Component diagram. Solid arrows are calls; dashed arrows are optional or data dependencies.

5.3 Class diagram

Fig. 4 shows the main classes. An `Instance` is composed of one `Graph` and a list of `Service` and `BatchTask` objects. A `PlacementContext` wraps an instance together with its all-pairs distance matrix and provides the objective and capacity checks used by every placement algorithm. All four heaps share one interface (`push`, `pop`, `decrease_key`), so Dijkstra and Prim can be run with any of them.

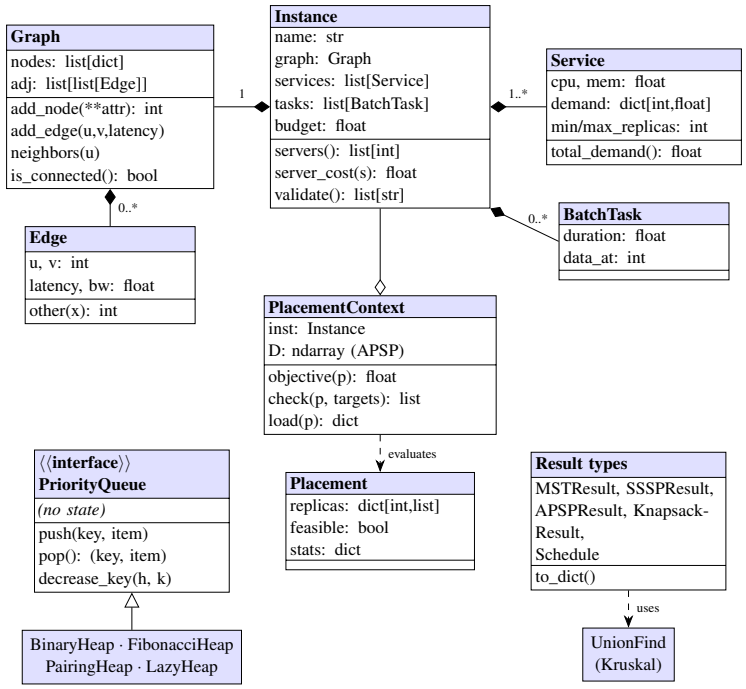


Figure 4: UML class diagram of the core model (filled diamond: composition; hollow diamond: aggregation; hollow triangle: realisation of an interface; dashed: dependency).

5.4 Sequence diagram for one solve request

Fig. 5 follows a request from the studio. The API builds an `Instance`; the solver computes the MST backbone and the all-pairs matrix, sizes replicas with the knapsack DP, runs every greedy (each followed by local search), runs the MILP when the instance is small enough, and assembles routes, schedules and recommendations into one JSON response.

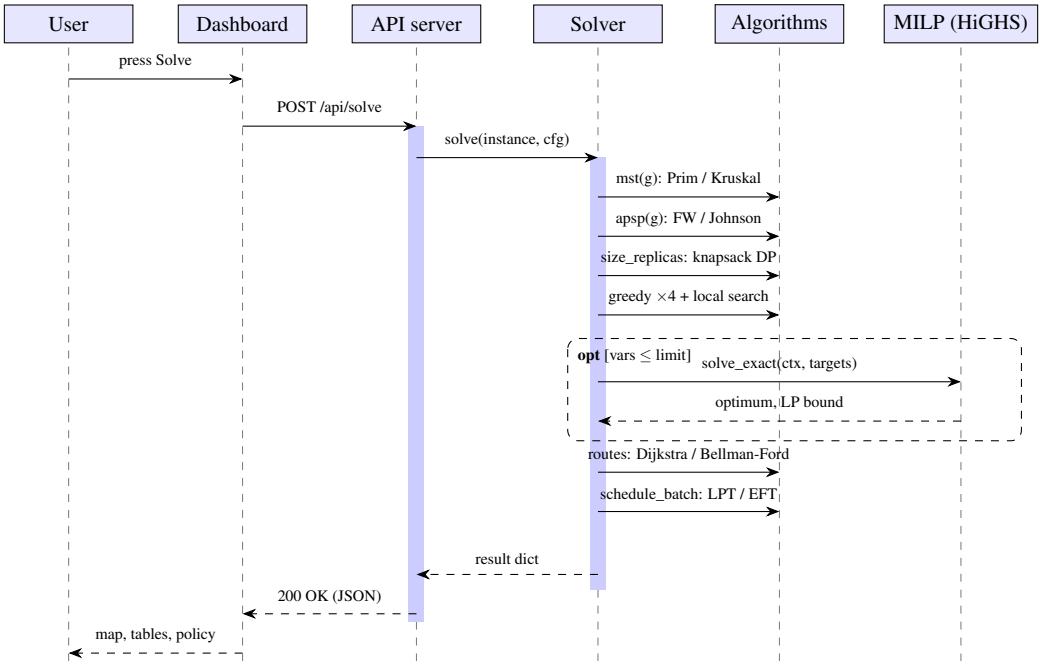


Figure 5: Sequence diagram of `POST /api/solve`. Dashed arrows are returns; the **opt** frame runs only when the MILP is small enough.

5.5 Activity diagram of the solver

Fig. 6 shows the decisions inside the solver. Two of them are made automatically from graph size: Prim or Kruskal for the backbone, and a dense or sparse all-pairs method. If the plan does not fit in capacity, the solver falls back to first-fit-decreasing packing before local search.

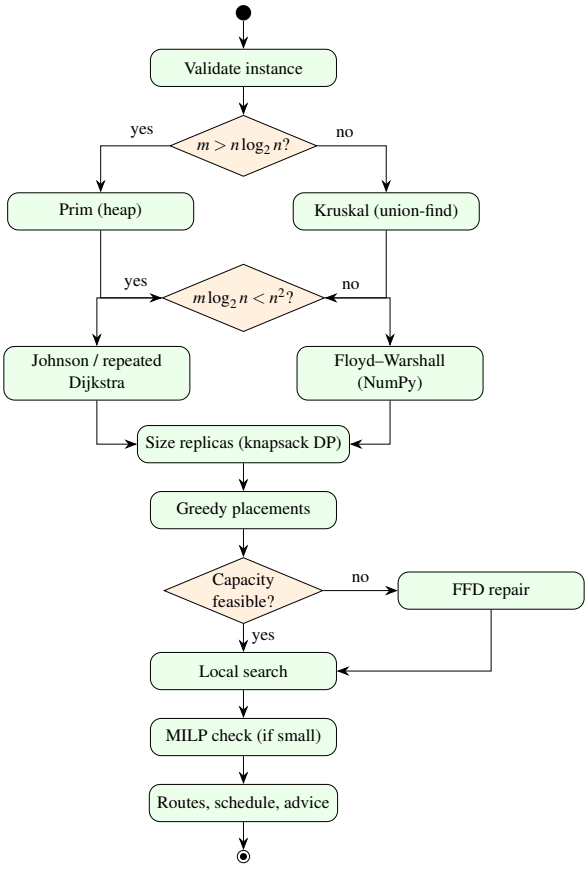
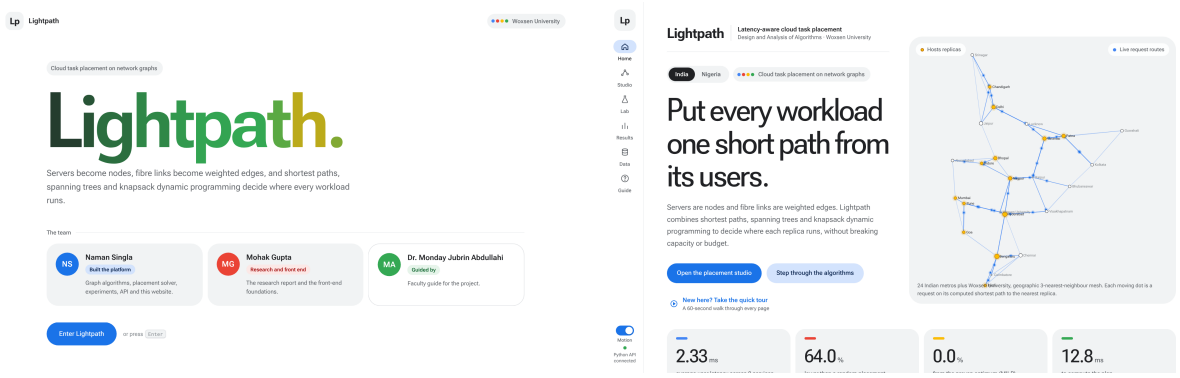


Figure 6: Activity diagram of the solver pipeline with its automatic algorithm choices.

6 Implementation and Working Screenshots

All screenshots were captured from the running build at 1440×900 using the live Python engine; the same build also runs offline on saved results.

Landing and Home. The landing page introduces the project and the team (Fig. 7a). Home shows a live map of India or Nigeria in which yellow cities host copies and moving dots are requests travelling their shortest path to the nearest copy, followed by four headline numbers (average latency, improvement over random, gap from the optimum, solve time), the seven-stage pipeline and the ten networks. A *quick tour* link starts a 13-step guided tour.



(a) Landing page with the team. **(b)** Home: live request map and headline numbers.

Figure 7: Landing and Home pages.

Placement Studio (Fig. 8). The user chooses a network, the VM budget, an optional server budget, the sizing method and the strategies to compare, then presses *Solve placement*. The map shows where

every service runs, with layers for the MST backbone, copies and routes. The right panel compares every strategy against the exact optimum and gives plain-language advice. Tabs show the route table, server load, the cost-benefit of each extra copy, replica-sync routes and the batch schedule.

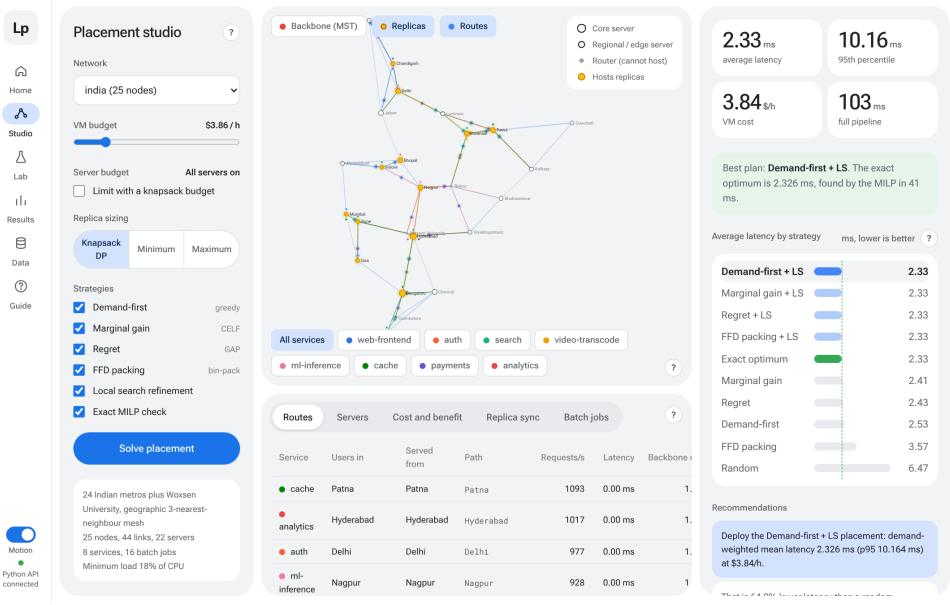
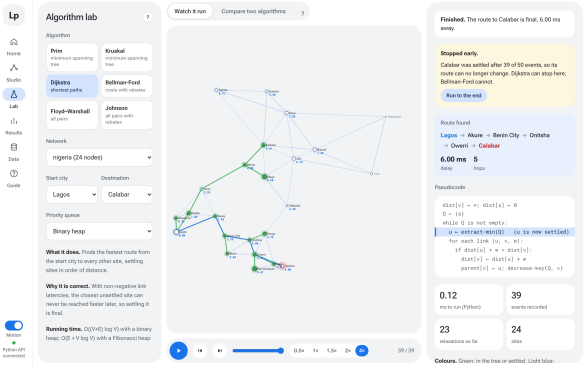
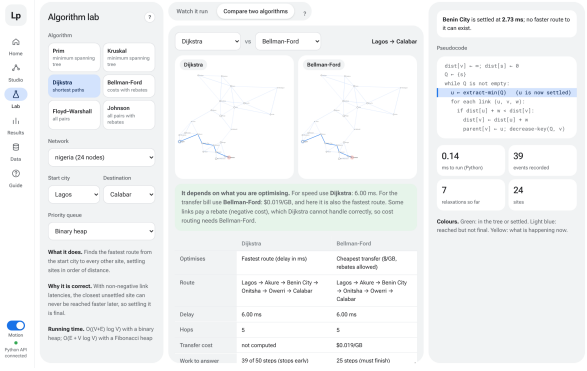


Figure 8: Placement Studio solving the India network.

Algorithm Lab (Fig. 9). Any of the six graph algorithms can be replayed step by step with its pseudocode highlighted. The user picks a **start city** and a **destination** and a speed from 0.5× to 4×. When the run ends, the route is drawn in blue and listed hop by hop with its delay and cost. Dijkstra and Prim stop early once the destination is final, which shows why Dijkstra suits a single trip; Bellman-Ford shows the red rebate arcs; Floyd–Warshall and Johnson show the full distance table as a heat map. **Compare two algorithms** runs two algorithms on the same trip, shows both routes side by side with a table of delay, hops, cost and work, and states which suits the trip best.



(a) Dijkstra from Lagos to Calabar, stopped early.



(b) Comparing Dijkstra and Bellman-Ford.

Figure 9: Algorithm Lab.

Results, Data and Guide. Results presents the eight experiments as interactive charts with one-line takeaways and CSV downloads. Data lists all ten networks with JSON, Gephi and CSV downloads and a generator for new random networks. The Guide explains every page in plain language, and every panel has a **?** help button.

Running it. In the project folder, `./run.sh` (macOS/Linux) or `run.bat` (Windows) installs the Python packages, builds the website and opens `http://localhost:8000`; `python -m pytest` runs the tests and `python experiments/run_experiments.py` the experiments. The source code is at

`https://github.com/namansingla-glitch/lightpath-cloud-task-placement`.

7 Complexity Analysis

Table 2 summarises the cost of each stage, using V for sites, E for links, W for the budget in cents, J for services, k for copies per service and S for servers. The dominant cost on large networks is the all-pairs latency matrix. On small networks with many services, greedy placement and local search dominate. The exact MILP is exponential in the worst case, which is why it serves only as a referee.

Table 2: Time complexity and guarantees by pipeline stage.

Stage	Algorithm	Time	Guarantee
Backbone	Prim (binary heap)	$O(E \log V)$	exact (cut property)
	Prim (Fibonacci heap)	$O(E + V \log V)$	exact
	Kruskal + union-find	$O(E \log E)$	exact
Routing	Dijkstra	$O((V+E) \log V)$	exact for non-negative weights
	Bellman-Ford	$O(VE)$	exact with negative edges; detects negative cycles
Latency matrix	Floyd–Warshall	$\Theta(V^3)$	exact
	Johnson	$O(VE + V(V+E) \log V)$	exact with negative edges
Activation	0/1 knapsack DP	$\Theta(SW)$	optimal for the given values
Sizing	Multiple-choice knapsack DP	$\Theta(W \cdot \sum \text{options})$	optimal for the given values
Placement	4 greedy schedulers	polynomial, about $O(JkSV)$	heuristic; $1 - 1/e$ for CELF without capacities
Refinement	Relocate/swap local search	polynomial per pass	local optimum; never worse than its input
Referee	MILP (HiGHS)	exponential worst case	proven optimum or lower bound
Batch	LPT, earliest finish	$O(T \log T + TS)$	within $\frac{4}{3}$ of optimal without transfers

8 Testing and Demonstrated Scenarios

8.1 Automated test suite

Correctness is checked by 191 automated tests (Table 3), all passing. Each algorithm is compared with an *independent* reference rather than only with our own expectations: graph algorithms against NetworkX on random graphs, knapsack DPs against brute-force enumeration, and the MILP against exhaustive search on small placements.

Table 3: Automated test suite (191 tests, all passing).

File	Tests	What is checked
`test_core.py`	15	Heap operations with random inputs (Hypothesis), union-find, graph construction
`test_paths.py`	66	MST, Dijkstra, Bellman-Ford, Floyd–Warshall and Johnson against NetworkX with every heap; negative-cycle detection; no negative cycle in any transfer graph
`test_optimisation.py`	100	Knapsack and multiple-choice knapsack against brute force; MILP against exhaustive search; greedies never exceed capacity; local search never makes a plan worse; scheduler lanes never overlap; the full pipeline on every dataset
`test_api.py`	10	Every API endpoint, error codes and the path-traversal guard

8.2 Scenario A: the full pipeline on India

On the India network (25 sites, 44 links, 8 services) the solver builds a 24-link MST backbone with a total latency of 56.2 ms, sizes the services within a $3.86/h budget and runs all four greedy schedulers with local search. The best plan reaches an average latency of **2.326 ms**, 64 % lower than a random placement, and it *matches the exact MILP optimum* (gap 0.00 %) while taking 4.1 ms against the MILP's 27 ms. Mumbai, Pune, Kochi, Chandigarh and Varanasi run at 90 % CPU or more, so capacity there is the binding constraint.

8.3 Scenario B: routes and negative costs

In the Lab on the Nigeria network, Dijkstra from Lagos to Calabar finds the route Lagos → Akure → Benin City → Onitsha → Owerri → Calabar (6.00 ms, five hops), settling the destination after 39 of 50 recorded events and stopping early. On India, the Compare view shows the same lesson for Chennai → Kochi: Dijkstra's route takes 3.93 ms, while travelling only along Prim's spanning tree takes 5.20 ms (1.32×). Requests are therefore routed on the full network, and only the backbone is provisioned from the MST.

On transfer costs, Bellman-Ford finds the cheapest Chennai → Kochi transfer to be −$0.025 per GB, a net rebate; the Compare view shows that Dijkstra cannot be trusted on these weights.

8.4 Scenario C: capacity pressure and infeasibility

On the `waxman-60-tight` network every server has half the normal capacity. As capacity shrinks further (Experiment E5), latency-first strategies stop finding complete plans: at 0.35× capacity, demand-first, marginal gain and regret fail in every run and are reported as *infeasible* rather than returned as broken plans. FFD packing still finds a valid plan at every level, so the solver uses it as a fallback. With capacity at 0.1× the solver reports the shortage and every result can still be read (a dedicated test checks this).

9 Experimental Evaluation

9.1 Method

All experiments call the Python engine directly, bypassing the website. Networks come from seeded generators so that results are reproducible. Timings are medians of repeated runs on a single core in CPython 3.13, and quality results average three seeds. The full suite runs in about 80 seconds and writes CSV files and the figures below. Table 4 lists the ten networks.

Table 4: The ten networks. Load is the CPU needed by the minimum copies divided by total capacity; best is the recommended plan's average latency.

Name	Source	Sites	Links	Servers	Services	Load	Best (ms)
india	real coordinates + Woxsen	25	44	22	8	18%	2.326
nigeria	real coordinates	24	47	21	8	23%	1.137
woxsen	campus map	23	29	13	8	17%	0.055
abilene	real topology	11	14	10	6	42%	3.827
nsfnet	real topology	14	21	12	7	26%	5.220
fat-tree-k4	data centre	36	48	16	6	18%	0.022
waxman-60	synthetic WAN	60	89	51	14	13%	5.245
waxman-60-tight	half capacity	60	89	51	14	27%	6.686
er-100	Erdős–Rényi	100	350	85	16	10%	5.062
ba-150	Barabási–Albert	150	296	128	20	7%	7.594

Abilene and NSFNET are real US research backbones. India (24 metros plus Woxsen University) and

Nigeria (24 cities, with Lagos, Abuja and Port Harcourt as core data centres) use real city coordinates with each city linked to its three nearest neighbours. The Woxsen campus has 23 buildings laid out from the official campus map. The synthetic families let us vary size and density: Waxman geographic networks [12], Erdős–Rényi random graphs, Barabási–Albert hub-heavy graphs [13] and a data-centre fat-tree.

9.2 E1: priority queues

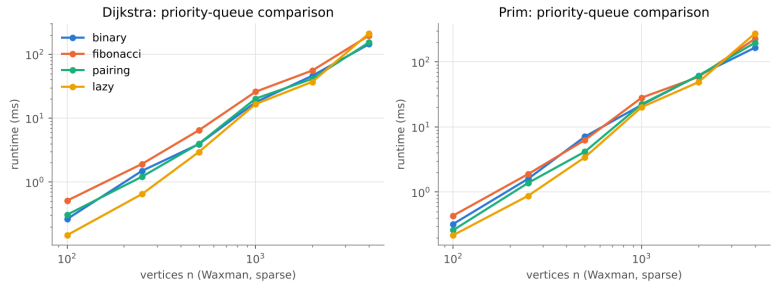


Figure 10: E1: Dijkstra and Prim with four priority queues on sparse graphs of up to 4000 sites.

The Fibonacci heap has the best Big-O bound but was consistently *slower* (Fig. 10). At 1000 sites Dijkstra took 25.9 ms with it against 17.7 ms with the binary heap, and at 4000 sites 196 ms against 144 ms. Its pointer-heavy nodes and consolidation cost more in an interpreter than the decrease-key operations it saves. Python's built-in lazy heap is fastest up to 2000 sites but falls behind at 4000 as stale entries accumulate.

9.3 E2: all-pairs shortest paths

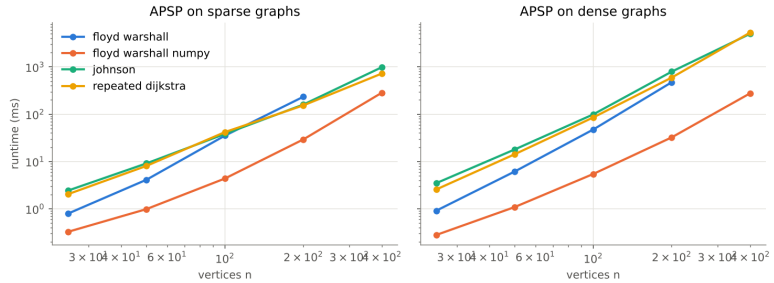


Figure 11: E2: all-pairs algorithms on sparse and dense graphs.

On dense graphs (Fig. 11) the vectorised Floyd–Warshall wins clearly: 277 ms at 400 sites against 5015 ms for Johnson and 5323 ms for repeated Dijkstra. On sparse graphs repeated Dijkstra (724 ms) beats Johnson (985 ms), but NumPy Floyd–Warshall is still faster (285 ms). The asymptotic rule correctly predicts that Dijkstra beats a pure-Python V^3 loop (234 ms already at 200 sites), but vectorisation moves the crossover well beyond 400 sites.

9.4 E3: spanning trees versus density

With 600 sites and 1.5k to 108k links (Fig. 12), Kruskal is fastest on sparse graphs (1.5 ms against 3.7 ms) and slowest on dense ones (126 ms against 42 ms), exactly as its sort-dominated cost predicts. Every variant returned the same tree weight.

9.5 E4: solution quality and cost versus size

This is the main quality experiment (Fig. 13, Table 5). Averaged over networks of 20 to 240 servers, greedy alone is 8.7 % (marginal gain), 9.2 % (regret), 16.7 % (demand-first) and 38.4 % (FFD) above the optimum, while a random placement is 167 % above. **Local search brings every method to about 0.5 % on average**, with a worst case of about 3 %. The exact MILP grows from 27 ms at 20 servers to 1.7 s at

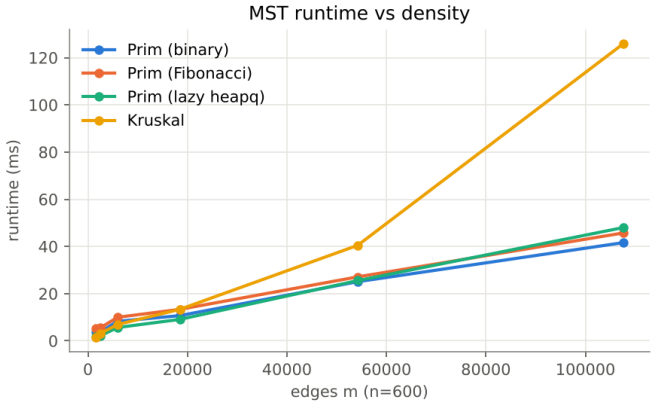


Figure 12: E3: Prim with four heaps against Kruskal as edge density grows (600 sites).

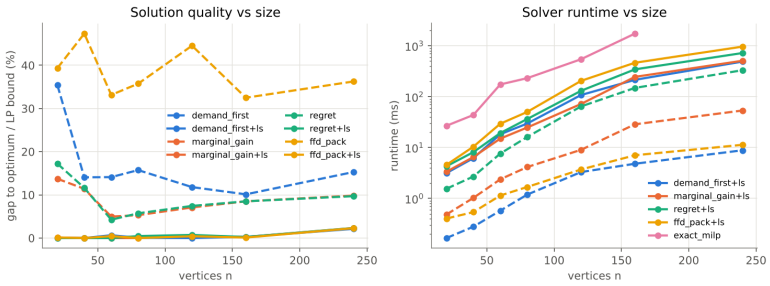


Figure 13: E4: distance from the optimum (left) and run time (right) as networks grow. Dashed lines are greedy alone; solid lines include local search.

160, against 4 ms and 344 ms for regret with local search, about five times cheaper. Greedy alone is 10 to 100 times cheaper than the MILP.

Table 5: E4: average distance from the MILP optimum (%) on Waxman networks, three seeds per size.

Strategy	20 servers	40	60	80	120	160
Random	186.63	163.01	165.61	137.93	166.80	172.12
FFD packing	39.35	47.34	33.14	35.75	44.54	32.50
Demand-first	35.41	14.06	14.11	15.73	11.80	10.12
Regret	17.23	11.61	4.24	5.73	7.44	8.51
Marginal gain	13.73	11.39	4.97	5.30	7.09	8.51
FFD packing + LS	0.14	0.00	0.44	0.00	0.41	0.11
Demand-first + LS	0.00	0.00	0.65	0.05	0.00	0.29
Regret + LS	0.00	0.00	0.00	0.45	0.72	0.24
Marginal gain + LS	0.14	0.00	0.00	0.00	0.52	0.24

9.6 E5: capacity pressure

At normal capacity every method finds a valid plan (Fig. 14). At 0.35× capacity, the three latency-first methods fail in every run because their early choices fragment server capacity, while FFD packing remains valid at every level and local search then brings its latency close to optimal. Under tight capacity, feasibility and latency pull in different directions, and packing first is the safer strategy.

9.7 E6, E7 and E8: density, budget and knapsack scaling

E6 (Fig. 15): raising link density from 0.025 to 0.7 cut the optimal average latency from 7.71 ms to 2.59 ms (67 %), with little extra gain beyond about 0.2. Regret plus local search matched the optimum at five of six densities. **E7**: on India, increasing the VM budget from $2.41 to $7.51 per hour bought 9 to 27

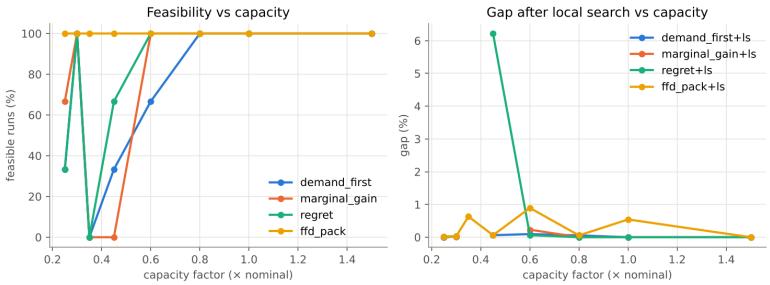


Figure 14: E5: share of runs with a valid plan (left) and distance from the optimum after local search (right) as server capacity shrinks.

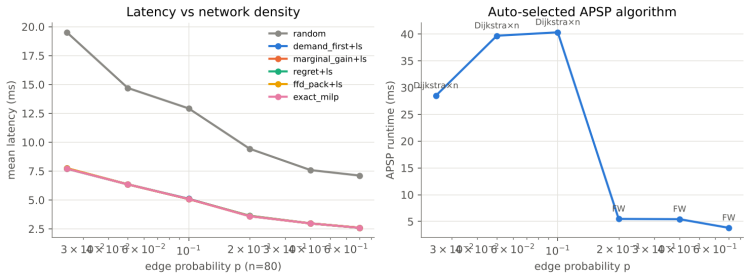
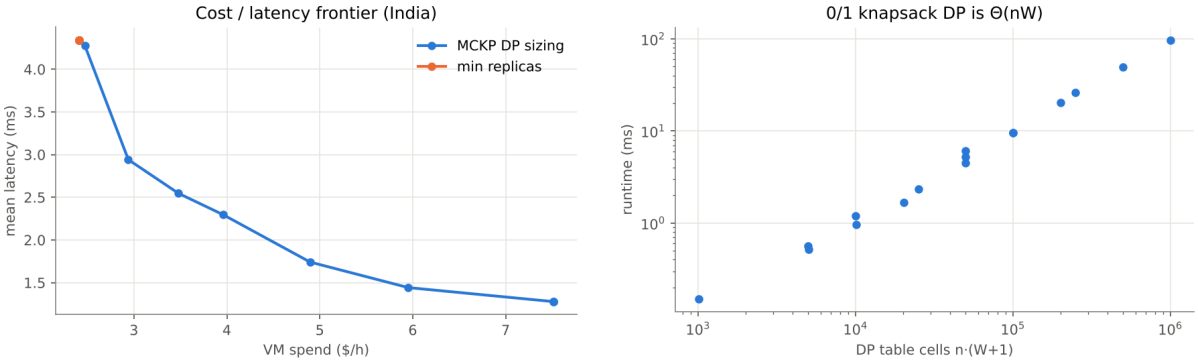


Figure 15: E6: optimal latency and heuristic gap as link density grows (80 sites).

copies and cut latency from 4.34 ms to 1.28 ms (70 %), with diminishing returns. **E8**: knapsack run time grew linearly with table size, about 97 ns per cell, confirming the $\Theta(nW)$ bound (Fig. 16).



(a) E7: budget against latency (India).

(b) E8: knapsack run time against table size.

Figure 16: Budget frontier and knapsack scaling.

Table 6: Summary of the eight experiments.

Exp.	Varied	Main finding
E1	size, heap type	Binary heap fastest at scale; Fibonacci slower in practice despite its better bound
E2	size, density	NumPy Floyd–Warshall wins on dense graphs and, at these sizes, on sparse ones too
E3	density	Kruskal best when sparse, Prim best when dense; identical tree weight
E4	size	Greedy + local search within about 0.5 % of the optimum, about 5× faster than the MILP
E5	capacity	Under tight capacity only FFD packing always finds a valid plan
E6	density	More links cut latency by 67 %, with diminishing returns after density 0.2
E7	VM budget	\$2.41 → \$7.51/h cuts latency by 70 % (4.34 → 1.28 ms)
E8	DP table size	Linear in table size, about 97 ns per cell

10 Case Studies and Policy Outputs

For every network the solver produces deployable *policies*: a route table (each city, its copy, the path and latency, and the stretch if only the backbone were used), a server schedule with the VMs and utilisation of each server, a cost-benefit table giving the latency saved by each extra copy and its price, the cheapest replica-synchronisation routes, and a batch-job timetable. Table 7 and Figs. 17–18 summarise the three main networks.

Table 7: Recommended plans. Stretch is the latency increase if requests used only the MST backbone.

Network	Mean (ms)	p95 (ms)	vs. random	Gap from optimum	Stretch	Best next copy
India	2.326	10.16	−64.0%	0.00 %	1.55×	video-transcode: −2.45 ms for $0.48/h
Nigeria	1.137	2.88	−44.3%	0.09 %	1.23×	analytics: −1.23 ms for $0.58/h
Woxsen	0.055	0.21	−38.9%	0.00 %	1.39×	dining POS: −0.028 ms for $0.06/h

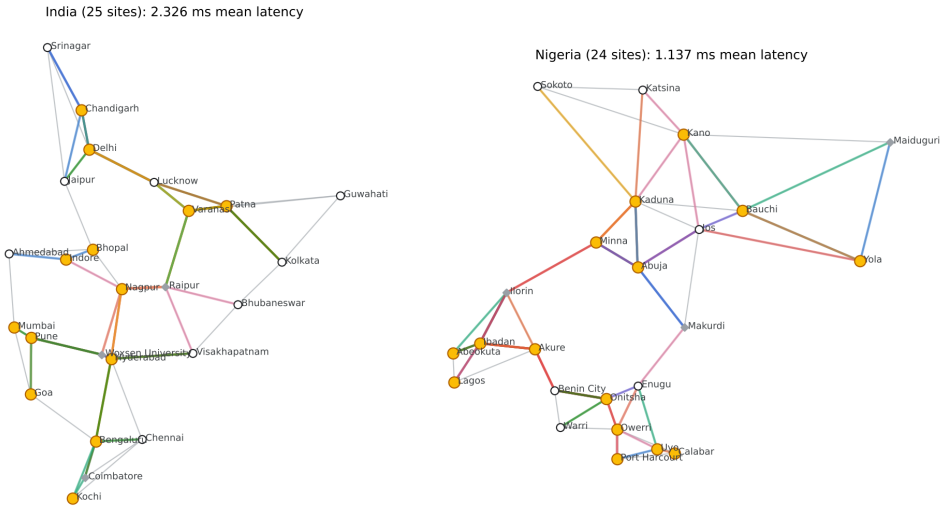


Figure 17: Recommended placements for India and Nigeria. Yellow sites host copies; coloured lines are request routes, one colour per service.

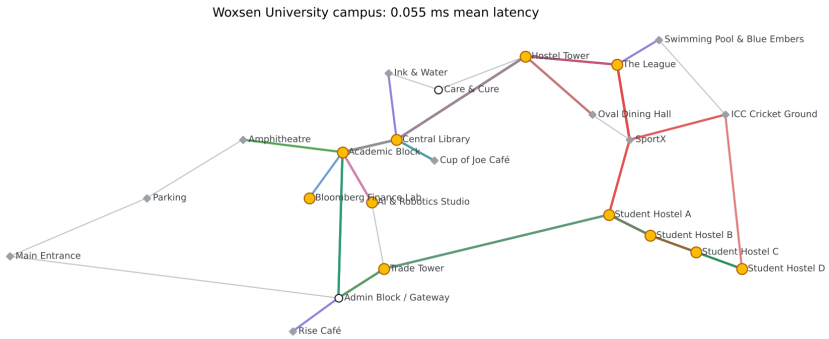


Figure 18: Woxsen University campus plan. Copies cluster around the Academic Block, the hostels and The League sports arena, where demand originates.

India. The best plan matches the MILP optimum; upgrading the saturated Mumbai, Pune, Kochi, Chandigarh and Varanasi servers is the most valuable hardware change. **Nigeria.** The MILP optimum is 1.137 ms and marginal gain with local search is within 0.09 % of it. Lagos, Abuja and Port Harcourt host the core VMs while Onitsha and Owerri saturate. **Woxsen.** Every building is within 0.52 ms. Hostel and sports traffic dominate, so the learning portal and Wi-Fi authentication replicate into Student Hostels A and D, and live streaming runs at The League.

Recommendations

1. **Default solver:** run all four greedy schedulers with local search and keep the best. This lands within 0.5 % of optimal on average, and the MILP can certify the result on smaller networks.
2. **Capacity first when load is high:** when servers are crowded, start from FFD packing, then refine for latency (E5).
3. **Spend by marginal value:** buy the next copy with the best milliseconds-per-dollar ratio from the cost-benefit table; returns diminish quickly (E7).
4. **Build the MST backbone, but route on the full network:** routing only along the tree would be 1.2–1.6× slower.

11 Correctness Reasoning (Proof Sketches)

MST (cut property). For any split of the sites into two groups, the cheapest crossing link belongs to some minimum spanning tree. Prim applies this to the split between tree and non-tree sites, Kruskal to the two groups a link joins, so both add only safe links and return a minimum spanning tree. **Dijkstra.** When the closest unsettled city u is taken, any other route to it must leave the settled region through a city at least as far away; with non-negative weights the rest of that route cannot shorten it, so u's distance is final [14]. This fails with negative weights, which is why transfer costs use Bellman-Ford. **Bellman-Ford.** After round i every route of at most i links is correct; loop-free routes have at most $V-1$ links, so $V-1$ rounds suffice, and any improvement in round V reveals a negative cycle. **Knapsack DPs.** An optimal choice either skips the last item (and is optimal for the rest at the same budget) or takes it (and is optimal for the rest at the reduced budget); the table tries both, so every entry is optimal by induction. **Local search.** Each accepted move strictly lowers latency and there are finitely many plans, so it stops, and it is never worse than its greedy start.

Lemma 1 *The transfer-cost network has negative arcs but no negative cycle.*

Proof sketch. Each arc costs a positive base plus a positive per-millisecond charge plus $\varphi(u) - \varphi(v)$, the difference of two regional price levels. Around any closed loop these differences cancel, so every loop's total is positive. The same "potential" idea makes Johnson's reweighting correct. □

12 Limitations and Future Work

India and Nigeria use real city locations but synthesised links, and Woxsen's fibre plan and server sizes are modelling assumptions; real topologies such as the Internet Topology Zoo could be added. Demand is static and latency ignores queueing at busy servers, so time-varying re-placement and queueing models (for example M/M/c) are natural next steps. Server activation optimises an estimated value per server rather than the joint problem. Timings reflect CPython; a compiled core would extend the exact referee to larger networks.

13 Conclusion

Lightpath shows how classical algorithm design fits a practical systems problem. Exact algorithms (Prim, Kruskal, Dijkstra, Bellman-Ford, Floyd–Warshall and Johnson) and two knapsack dynamic programs handle the tractable parts. Greedy schedulers with local search handle the NP-hard placement, and an exact solver confirms that they land within about 0.5 % of the optimum on average while running about five times faster. The experiments show where Big-O and measured speed agree and where they do not, and the same pipeline produces deployable plans for India, Nigeria and the Woxsen campus.

References

[1] O. Kariv and S. L. Hakimi, "An algorithmic approach to network location problems. II: The p-medians," *SIAM Journal on Applied Mathematics*, vol. 37, no. 3, pp. 539–560, 1979.

[2] M. L. Fredman and R. E. Tarjan, "Fibonacci heaps and their uses in improved network optimization algorithms," *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, Jul. 1987.

[3] R. E. Tarjan, "Efficiency of a good but not linear set union algorithm," *Journal of the ACM*, vol. 22, no. 2, pp. 215–225, Apr. 1975.

[4] R. C. Prim, "Shortest connection networks and some generalizations," *Bell System Technical Journal*, vol. 36, no. 6, pp. 1389–1401, Nov. 1957.

[5] J. B. Kruskal, "On the shortest spanning subtree of a graph and the traveling salesman problem," *Proceedings of the American Mathematical Society*, vol. 7, no. 1, pp. 48–50, 1956.

[6] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[7] R. Bellman, "On a routing problem," *Quarterly of Applied Mathematics*, vol. 16, no. 1, pp. 87–90, 1958.

[8] R. W. Floyd, "Algorithm 97: Shortest path," *Communications of the ACM*, vol. 5, no. 6, p. 345, Jun. 1962.

[9] D. B. Johnson, "Efficient algorithms for shortest paths in sparse networks," *Journal of the ACM*, vol. 24, no. 1, pp. 1–13, Jan. 1977.

[10] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, "An analysis of approximations for maximizing submodular set functions—I," *Mathematical Programming*, vol. 14, no. 1, pp. 265–294, 1978.

[11] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018.

[12] B. M. Waxman, "Routing of multipoint connections," *IEEE Journal on Selected Areas in Communications*, vol. 6, no. 9, pp. 1617–1622, Dec. 1988.

[13] A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," *Science*, vol. 286, no. 5439, pp. 509–512, Oct. 1999.

[14] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.